

Components of Convolutional Neural Networks

March 2026

Contents

1	Background	2
1.1	History	2
1.2	Terms	2
2	Dot Products	2
2.1	Sliding Dot Product	2
2.2	1D Sliding Dot Products	2
2.3	2D Sliding Dot Products	3
3	Unsqueeze	3
4	1d MaxPool	3
5	Variance and Standard Deviation	3
6	1d BatchNorm	4
7	Flatten	4

1 Background

1.1 History

Convolutional Neural Networks (CNNs) are the machine learning algorithms that can label what an image is. You know how when you take a picture of a plant or animal species you don't know, and you swipe up on the photo on an iPhone, it shows you what type of species you took a picture of. That is a CNN at work on your iPhone. However, this technology has existed for a while now. Did you know as early as the 1980's UPS used CNNs to sort packages? Fast forward to 2012 CNNs advanced a lot, leading to the AI revolution kicking off.

1.2 Terms

Here are some terms I will use. The first word is PyTorch. PyTorch is the one of the best Python libraries for machine learning out there. I will be using mostly PyTorch when I code. The second word is Tensor. Tensor is a fancy word for arrays, which are just lists of numbers.

2 Dot Products

What's a Dot Product? A Dot Product is a mathematical equation where you take two Tensors. We can set two lists up.

```
1 a = [2,3]
2 b = [4,6]
```

Then we multiply the numbers that are in the same position with each other.

```
1 a[0] * b[0] # This is 2 * 4
2 a[1] * b[1] # this is 3 * 6
```

Then add up the multiplied numbers and you have your dot product. In our example that would be $2 * 4 = 8$ and $3 * 6 = 18$. Then $8 + 18 = 26$. This would be our dot product.

2.1 Sliding Dot Product

What's a Sliding Dot Product? A sliding dot product is just like a dot product, but it moves. This is important for machine learning, as it filters through the data and compresses it until the machine learning model is eventually able to read it. [This website includes really cool animations of sliding dot products.](#) [This website has a really good visualization of how CNNs work.](#)

2.2 1D Sliding Dot Products

My code for 1D sliding dot products.

```
1 """
2 Demonstrating sliding dot products IE 1d convolutions
3
4 Answers checked manually, this is correct code
5 """
6
7 # 1d convolution
8 import torch
9 X = torch.tensor([1.0,2,3,4,5])
10 h = torch.tensor([0.5,0.5]) # This is our filter or kernal
11 width = X.shape[0]
12
13 output = torch.zeros(width-1) # We are doing width-1 because the gliding for the dot product essential
14 for i in range(len(X)-1): # now we are looping over our data
15     output[i] = torch.dot(X[i:i+2],h) # this whole thing is a sliding dot product, aka a convolution
16 print(output)
```

2.3 2D Sliding Dot Products

My code for 2D sliding dot products.

```
1 """
2 Demonstrating sliding dot products IE 2d convolutions
3
4 Answers checked manually, this is correct code
5 """
6
7 # 2d convolution
8
9 X = torch.tensor([[1,2,3,4],[5,6,7,8],[9,10,11,12]],dtype=torch.float32)
10 h = torch.tensor([[0.5,0.5],[0.5,0.5]])
11 height,width = X.shape
12 output = torch.zeros(height-1,width-1)
13 print(height,width)
14 for i in range(height-1):
15     for j in range(width-1):
16         #print(i,j)
17         #print(X[i:i+2,j:j+2])
18         output[i,j] = torch.sum(X[i:i+2,j:j+2]*h)
19
20
21 output
```

3 Unsqueeze

Unsqueeze adds a dimension to the array each time you run it.

```
1 h = torch.tensor([0.5,0.5])
2 h.unsqueeze(0).unsqueeze(1) # Because I ran this command twice, it unsqueezed h to 3 dimensions
3 # h now equals [[[0.5,0.5]]]
```

4 1d MaxPool

Essentially, MaxPool is a sliding dot product. [This website includes really cool animations of sliding dot products.](#) When you use MaxPool, you have to specify its kernel size, and dilation. The kernel size is how much of the tensor you want to select at a time, and the dilation is how spread apart do you want each point to be in the kernel. For example, if you have a kernel size of 3 and a dilation of 1 it will select 3 numbers at a time that are close to each other. But if you have a kernel size of 2 and a dilation of 2 it will select 2 numbers that are 1 apart from each other. Then, you put a tensor into the MaxPool. The MaxPool then takes a selection of the tensor using your specified kernel size and dilation, and it chooses the maximum (max) number of the selected numbers. This is why it is called MaxPool.

5 Variance and Standard Deviation

Variance measures how far out numbers are from their average. The standard deviation is the square root of the variance. For example, if you take the standard deviation of a tensor that contains the numbers 1, 2, 3, you will get 0.8165. [You can read more on variance and standard deviation here.](#)

```
X = torch.tensor([1,2,3],dtype=torch.float32)
torch.std(X,correction=0)
```

```
tensor(0.8165)
```

Figure 1: Standard Deviation Code and Answer

¹Note: The "correction = 0" just means that you're using high school standard deviation, not college level standard deviation.

6 1d BatchNorm

BatchNorm normalizes numbers in a tensor down columns. This is to keep the machine learning model from being confused, as ML models think that larger numbers are more important. BatchNorm shrinks the big numbers so that ML models treat all data fairly.

```
before: the right side has lots of large numbers
tensor([[ 1., 120.],
        [ 2., 110.],
        [ 3., 120.]])
after: the right sides large numbers have been normalized
tensor([[ -1.2247,  0.7071],
        [ 0.0000, -1.4142],
        [ 1.2247,  0.7071]], grad_fn=<NativeBatchNormBackward0>)
```

Figure 2: Batchnorm Effects on Datasets

7 Flatten

Flatten is just like Unsqueeze, but in the opposite direction. It removes dimensions from tensors, instead of adding them.